

Synergy Network - DAG Validator Cluster Integration and DAG Security Enforcement Plan

Grounded in the attached Synergy specifications and roadmap. Repository-specific workstream mapping is included, but exact code-level repo audit could not be completed in-session because the two GitHub repositories were not retrievable from this environment.

Prepared: 2026-03-19

Important scope note. This report is fully grounded in the uploaded Synergy protocol, node, network, security, wallet, rewards, naming, RPC, and roadmap documents. The two referenced repositories could not be fetched in-session from this environment, so the repository recommendations below are an inferred workstream map rather than a file-by-file code audit. That limitation matters because the actual validator/consensus engine changes may live in a third repo that was not available here.

1. Executive conclusion

Synergy should **not** replace Proof of Synergy finality with a pure DAG-finality network. The attached PoSy materials define a clustered, Byzantine-fault-tolerant, quorum-certificate-based protocol with **deterministic finality, dual quorum enforcement, epoch rotation**, and an explicit **safety-over-liveness posture**. The roadmap already points to **hybrid DAG execution within validator clusters**, which is the correct interpretation: use a DAG inside clusters to improve data availability, dissemination, batching, and pre-ordering, while keeping PoSy quorum certificates as the only canonical finality artifact.

The right architecture is therefore a **cluster-local certified DAG**. Validators create signed DAG vertices containing encrypted transaction batches or references to those batches. Peers acknowledge availability. Only **certified** vertices become eligible for proposer ordering. The proposer builds a block from a deterministic DAG cut, validators verify the cut and the resulting state transition, and PoSy's existing **dual-quorum** logic still produces the final QC.

That approach is consistent with modern DAG/BFT practice. Narwhal/Tusk-style designs explicitly separate **reliable transaction dissemination** from **transaction ordering**, and Bullshark-style work shows why a DAG can be an efficient data plane or consensus substrate without forcing the system into a fully probabilistic, tip-selection-driven security model. Synergy's own documents make clear that finalized state must stay unambiguous, externally verifiable, and resistant to fork ambiguity; a cluster-local DAG complements that, while a pure DAG-finality pivot would cut across

the grain of the current architecture. [Source note: PoSy_updated.docx extracted lines 392-505 and 513-615; 03-proof-of-synergy-consensus.md extracted lines 95-147 and 555-645.]

From a security standpoint, the biggest design mistake would be to bolt a high-throughput DAG onto the validator clusters while weakening the controls that already distinguish Synergy's design: authenticated cluster bridges, replay resistance via timestamps and sequence numbers, anti-cartel scoring, quarantine on invariant failure, encrypted mempool, signed release channels, audit-proof generation, and explicit refusal to continue optimistically when correctness is in doubt. DAG rollout must inherit those controls, not bypass them.

External reference note. The hybrid recommendation is also supported by public DAG/BFT literature showing that Narwhal separates dissemination from ordering and that hybrid DAG-plus-BFT models are a major direction for scalable secure systems, while DAG literature also documents challenges around ordering ambiguity, probabilistic finality, Sybil attacks, parasite-chain-style attacks, and double-spending if the graph itself is treated as the only finality mechanism. Public reference basis for this point: 'Narwhal and Tusk: A DAG-based mempool and efficient BFT consensus'; 'Bullshark: DAG BFT Protocols Made Practical'; and recent DAG security survey literature.

2. What the attached Synergy specifications already imply

Finding	Practical implication
PoS is already a clustered BFT system. Validators are grouped into cooperative clusters, run propose-validate-commit rounds, and finalize with quorum certificates.	Do not redesign Synergy as if it were an unconstrained, global, permissionless DAG. The cluster is the natural boundary for DAG integration.
PoS is deterministic and fork-averse. The documentation states that finalized blocks are unambiguous, that forks are pathological rather than normal, and that no block is preferable to a wrong block.	DAG must be a throughput and availability layer, not the sole source of finality.
The roadmap already names the target. The roadmap milestone explicitly says 'DAG Validator Cluster Architecture' and 'Deploy hybrid DAG execution within validator clusters.'	The architecture decision should be formalized, not rediscovered.
Node operations are intentionally bounded. The Node Technical Specification defines the Control Panel as the official no-code control plane, with role overlays, secret isolation, signed updates, quarantine flows, and explicit refusal of unsafe combinations.	Any DAG feature exposed to operators must be a bounded profile, not a raw free-form config surface.
Encrypted mempool is already a first-class security requirement. The security spec	The DAG data model should carry encrypted batches from day one, or the system will create

requires threshold encryption with decryption only at consensus ordering time.	a new ordering-manipulation surface just before Synergy intends to remove one.
Current beta/development parameters do not fully match the broader PoSy specification. The devnet/network configuration sets validator_cluster_size = 7 and max_validators = 15, while PoSy's broader architecture text describes clusters of 10-50 with a typical target around 30.	You need an explicit beta-versus-target ADR before implementation starts.

2.1 Canonicalization gaps that must be resolved before implementation

Gap	What the docs currently say	Why it matters	Required decision
Cluster size	PoSy_updated.docx: clusters of 10-50, typical target 30. Network configuration and master network docs: validator_cluster_size = 7, max_validators = 15.	DAG thresholds, parent diversity rules, and certificate math change with committee size.	For testnet-beta, keep 7 as the implementation proving ground, but document it as beta topology , not as the network's final architectural target.
Epoch duration	PoSy_updated.docx: epochs are typically about 384 slots / about 2 hours. Network configuration: consensus.epoch_length = 30,000 blocks with 5-second block time (roughly 41.7 hours).	Epoch cadence affects rotation security, rekeying, checkpoint frequency, and the blast radius of a compromised cluster.	Resolve this immediately. A 41.7-hour epoch is too long if cluster rotation is supposed to be a security primitive.
DAG scope	Roadmap says hybrid DAG execution within validator clusters, but the existing PoSy documents do not yet define DAG vertex, certificate, checkpoint,	Without a formal object model, implementations will diverge and auditability will suffer.	Create a protocol ADR and schema pack before code begins.

	or ordering-cut objects.		
RPC and naming support	Current RPC methods cover validators and general chain operations but not DAG observability; naming standards do not yet define DAG object prefixes.	Operators and auditors will be blind unless DAG artifacts become first-class protocol objects.	Add DAG RPCs and extend naming/encoding standards as part of the same release.

3. Recommended target architecture

The recommended design is a **hybrid, cluster-local, certified DAG** with unchanged PoSy finality. The DAG becomes the validator cluster's high-throughput data plane and pre-ordering plane. PoSy quorum certificates remain the finality plane. Cross-cluster communication continues to move through checkpointed artifacts and authenticated bridge relays, not through a network-wide raw DAG flood.

Element	Recommended design
Current flow	Transactions -> flat mempool -> proposer assembles block -> validators vote -> QC finalizes.
Target flow	Transactions -> encrypted batch -> DAG vertex -> availability receipts -> DAG certificate -> proposer selects deterministic DAG cut -> validators verify cut + state transition -> QC finalizes.
Finality source	Still the PoSy quorum certificate. The DAG never becomes final by itself.
Cross-cluster scope	Checkpoint/QC exchange only. Raw DAG replication stays intra-cluster.
Security model	Dual quorum, Synergy Score weighting, cooperation count threshold, anti-cartel logic, replay protection, signed updates, quarantine, and audit proofs stay in force.

3.1 Architecture decisions that should be locked immediately

1. **ADR-001 - Finality stays QC-based.** DAG improves the data plane and proposer input set. It does not replace dual-quorum commitment.
2. **ADR-002 - DAG scope is cluster-local.** Cross-cluster traffic carries checkpoints, aggregated proofs, and ledger commitments, not unconstrained vertex gossip.
3. **ADR-003 - Availability before ordering.** Only vertices with proven data availability may influence ordering.
4. **ADR-004 - Weighting belongs to finality, not existence.** Use committee membership and availability certification to determine whether a vertex exists; keep Synergy Score weighting in the finality path, not in the basic data-availability path.
5. **ADR-005 - Epoch boundaries are hard graph boundaries.** Previous-epoch vertices cannot bleed forward indefinitely; epoch rotation, rekeying, and watermarks must reset graph context cleanly.
6. **ADR-006 - Parallel execution is a second-stage feature.** First stabilize the dissemination DAG and deterministic ordering cut. Only then add an execution DAG for parallel scheduling.

Source note. These decisions align with the PoSy materials' repeated emphasis on rotating committees, deterministic finality, epoch-scoped responsibilities, authenticated bridge relays, and externally verifiable quorum material. See PoSy_updated.docx extracted lines 168-205, 392-505, and 513-615; 03-proof-of-synergy-consensus.md extracted lines 95-147, 175-241, and 521-713; Synergy_Network_Roadmap (1).docx extracted lines 137-153.

4. Detailed step-by-step implementation plan

Phase 0 - Governance freeze, scope control, and ownership mapping

Before anyone writes validator DAG code, Synergy needs a canonical architecture pack. Right now the attached specifications are directionally aligned but not perfectly canonicalized for DAG rollout. This phase eliminates ambiguity and prevents three teams from implementing three different 'hybrid DAG' concepts.

1. Publish a short **DAG integration ADR set** that makes four points explicit: (a) finality remains QC-based, (b) DAG is cluster-local, (c) availability certification is mandatory, and (d) deterministic execution DAG is deferred until after dissemination DAG is stable.
2. Resolve the **cluster-size mismatch**. For beta, adopt the existing `validator_cluster_size = 7` environment as the proving topology because the network configuration and machine matrix already assume it. Document that choice as a **beta topology**, not as the long-horizon architectural ceiling.
3. Resolve the **epoch-length mismatch**. Because PoSy treats rotation as a security primitive, do not carry a roughly 41.7-hour epoch into DAG beta without explicit justification. Shorter epochs materially reduce predictability and fault blast radius.

4. Identify the **actual consensus-engine repository**. A hard truth: DAG integration is unlikely to live fully inside a testnet config repo and a control-panel repo. If the validator client or consensus engine lives in a third repo, that repo becomes the primary implementation track.
5. Freeze the **security invariant list** that every later phase must preserve: no uncertified ordering, no weakening of QC verification, no new public attack surface by default, no plaintext key workflows, and no optimistic continuation after invariant failure.

Deliverables and exit criteria: Approved ADR package; architecture owner assigned; consensus-engine repo identified; beta topology and epoch plan signed off; invariant checklist ratified.

Source note. PoSy_updated.docx extracted lines 168-205, 392-505, 513-615;

Synergy_Network_Network_Configuration_Specification.docx extracted lines 2749-2819;

Synergy_Network_Node_Technical_Specification.docx extracted lines 2639-2759 and 2916-2930.

Phase 1 - Protocol objects, schemas, and interface contracts

This phase turns 'hybrid DAG' from a slogan into a formal protocol surface. The main objective is to create unambiguous objects that can be validated, signed, stored, audited, and exposed over RPC.

1. Define a canonical **DagVertex** object with at least: ``vertex_id``, ``epoch_id``, ``cluster_id``, ``round``, ``author_validator_id``, ``parent_vertex_ids``, ``encrypted_batch_root`` or ``batch_root``, ``batch_blob_ref``, ``prev_author_vertex_id``, ``timestamp``, ``sequence_number``, and ``author_signature``.
2. Define an **AvailabilityReceipt** object containing the receiving validator, the vertex commitment, receipt timestamp, local blob availability status, and ML-DSA signature.
3. Define a **DagCertificate** object that aggregates availability receipts and records at minimum the signer bitmap, signer count, signer set root, and certificate signature material. For beta, make the certificate threshold identity/count-based rather than Synergy-Score-weighted so the data plane does not become economically brittle.
4. Define an **OrderingCut** object that commits to the anchor vertex, the included certified-vertex set root, the deterministic topological order root, and the resulting transaction list root that feeds block construction.
5. Define a **ClusterCheckpoint** object that binds the most recent QC, DAG watermark, certificate root, and any cross-cluster summary material that bridge relays must forward.
6. Extend the **naming and encoding standard** so DAG objects receive typed user-facing prefixes and canonical hash domains. Do not let DAG identifiers appear in ad hoc forms across services.
7. Extend the **RPC contract** so operators and auditors can query DAG state, certificates, frontier health, ordering cuts, equivocations, and availability lag.

Deliverables and exit criteria: Schema package for all DAG objects; RPC spec delta; naming/encoding delta; deterministic ordering specification; signer and verifier test vectors.

Source note. Synergy_Protocol_Naming_Encoding_Standards.docx extracted lines 292-308 and 1649-1691; rpc-methods.md extracted lines 167-274; PoSy_updated.docx extracted lines 482-492 and 560-615.

Phase 2 - Build the cluster-local DAG availability layer inside the validator engine

This is the first real protocol implementation phase. The objective is to give each validator a way to turn incoming transactions into certified DAG vertices that are visible and recoverable across the cluster before any proposer uses them.

1. Add a **batch builder** inside the validator path. Transactions should be grouped into bounded batches by size, age, and conflict profile. Batches should carry encrypted payloads or encrypted-payload references if encrypted mempool is enabled, which it should be for this rollout.
2. Implement a **vertex builder** that packages the batch commitment, parent references, author chain link, round metadata, and envelope data; signs the vertex with ML-DSA; and sends it to cluster peers over the existing authenticated intra-cluster channel.
3. Enforce a **one-author-one-vertex-per-round** rule for the beta path unless there is a very strong reason to relax it later. This dramatically simplifies equivocation detection.
4. Implement **parent selection rules** that require parent diversity from the immediately preceding certified or near-certified round. Prefer parents from distinct authors. Do not allow a clique of validators to reference only one another when broader parent availability exists.
5. Require peers to **persist or fetch the full batch blob before acknowledging availability**. A receipt without actual availability is worse than no receipt.
6. Issue a **DagCertificate** only when the availability threshold is met. At cluster size 7, the natural beta threshold is 5 receipts ($2f+1$). As committees change size, compute the threshold from the BFT fault model rather than hard-coding magic numbers.
7. Store the vertex, receipts, and certificate in a separate DAG store rather than overloading the canonical chain-state store. Separation improves rollback discipline and makes archive/audit retention easier.

Deliverables and exit criteria: Cluster-local DAG propagation works under load; missing-parent fetch path works; certificates are produced deterministically; same-round equivocation becomes machine-detectable; batch blobs are durably retrievable.

Source note. PoSy_updated.docx extracted lines 168-205, 204-205, 409-449, and 586-613;

Synergy_Network_Security_Implementation_Specification.docx extracted lines 67-70 and 1370-1374.

Phase 3 - Integrate DAG ordering with PoSy block proposal and commitment

This phase is where the DAG becomes useful rather than merely interesting. The proposer stops reading from a flat mempool and instead constructs block candidates from a deterministic, certified slice of the cluster DAG.

1. Change the proposer so it selects a **safe anchor** and computes a deterministic **ordering cut** from the highest certified frontier it can justify locally.
2. Define one canonical **ordering rule** and enforce it everywhere. A strong beta rule is: topological order first; within the same dependency level sort by round, then by author-validator ID, then by vertex hash; within each vertex, preserve a deterministic transaction order.

3. Require the proposer to include in the block header or proposal body the **ordering-cut commitment**, the set root of the included certified vertices, and the batch/data commitments needed for independent validator verification.
4. Change validator verification so peers check not only transaction validity and state transitions, but also: certificate validity, parent consistency, absence of uncertified inputs, deterministic ordering correctness, and agreement between the block's transaction root and the ordering cut.
5. Keep the existing **dual-quorum commit path unchanged**. Validators still vote on the block candidate; the QC is still the finality artifact; and the network still refuses to finalize if quorums are not safely reached.
6. When a proposer references uncertified or unavailable vertices, treat that as either a malformed proposal or provable misbehavior depending on severity and intent. The beta path should be strict: incorrect DAG references are rejected, not tolerated.

Deliverables and exit criteria: Blocks can be independently reconstructed from DAG evidence; validators converge on the same transaction order; no uncertified data influences finality; PoSy QC verification remains intact.

Source note. PoSy_updated.docx extracted lines 441-505; 03-proof-of-synergy-consensus.md extracted lines 521-645.

Phase 4 - Cross-cluster checkpointing, bridges, and epoch transitions

Synergy already has a cluster-and-bridge worldview. The DAG rollout should respect it. Raw DAG traffic should remain local to the cluster, while cross-cluster traffic should carry compact checkpoint and proof objects.

1. Use **ClusterCheckpoint** packages to export the latest QC, DAG watermark, certificate root, and other minimal cross-cluster commitments. Do not fan out the full vertex graph across the whole network.
2. Keep inter-cluster relay traffic on the existing **authenticated bridge** model with ML-DSA message signatures, ML-KEM session protection, timestamps, sequence numbers, relay rate limits, and bridge redundancy.
3. At every epoch boundary, enforce a **hard graph boundary**: stop accepting new prior-epoch vertices, finalize any safe outstanding work, roll forward the checkpoint, rotate cluster membership, and rekey bridge sessions.
4. Make previous-epoch leftover vertices explicitly expire. Anything not certified and checkpointed by the epoch transition should either be discarded or resubmitted in the new epoch through a new vertex envelope.
5. Because epoch duration is presently inconsistent across the attached specs, treat checkpoint cadence and epoch cadence as part of the same architectural decision rather than separate tuning knobs.

Deliverables and exit criteria: Cross-cluster communication remains bounded and auditable; relay path preserves existing PQ protections; old graph data cannot silently leak into new epochs; checkpoint proofs can be reconstructed by audit nodes.

Source note. PoSy_updated.docx extracted lines 192-205, 513-615; 03-proof-of-synergy-consensus.md extracted lines 193-241; Master Network, Ports, and Nginx Configuration Manual.docx extracted lines 162-174.

Phase 5 - Add a deterministic execution DAG only after dissemination DAG is stable

The roadmap's wording around 'hybrid DAG execution' is best implemented in two stages. Stage one is the dissemination and ordering DAG described above. Stage two, only after the first path is stable, is a deterministic execution DAG that permits parallel execution of non-conflicting transactions.

1. Build a **dependency extractor** that derives conservative conflict classes from accounts, storage keys, nonces, or other state-touch metadata. Conservative is the right word here; false positives are acceptable, false negatives are dangerous.
2. Construct a per-block **execution DAG** from the already ordered transaction list. The ordered list remains canonical; the execution DAG only permits safe parallel scheduling where conflicts do not exist.
3. Require all validators to use the exact same dependency extraction and scheduling rules. If any ambiguity exists, fall back to serial execution for the ambiguous subset.
4. Never allow speculative execution to alter canonical order. Parallel execution is an optimization, not a new source of consensus truth.
5. Gate this entire phase behind a feature flag. It should not ship in the same blast radius as the first certified DAG rollout unless the engineering team has unusually strong confidence and unusually strong test coverage.

Deliverables and exit criteria: Parallel execution works without changing final state roots; nondeterminism is provably absent; serial fallback is automatic when dependency confidence is insufficient.

Source note. This phase is an implementation recommendation that interprets the roadmap's 'hybrid DAG execution' milestone conservatively, in line with PoSy's deterministic and safety-first design. See Synergy_Network_Roadmap (1).docx extracted lines 143-153; 03-proof-of-synergy-consensus.md extracted lines 541-567 and 627-645.

Phase 6 - Storage, archive retention, audit reconstruction, and evidence handling

A DAG that cannot be reconstructed is not secure enough for a validator network. This phase makes DAG behavior auditable after the fact, not merely observable in the moment.

1. Split persistence into at least four logical stores: canonical chain state, DAG metadata, batch/blob storage, and evidence/audit artifacts.
2. Allow live validators to **prune aggressively only after finality depth and checkpoint depth are safely crossed**. Archive validators should keep the full DAG history. Audit validators should keep enough material to reconstruct every ordering cut and every equivocation proof.
3. Emit **signed audit receipts** whenever a node accepts a certificate, finalizes an ordering cut, prunes a graph segment, or quarantines itself after an invariant failure.
4. Expose a machine-readable **evidence bundle** that includes vertex envelopes, receipts, certificates, ordering-cut commitments, block references, and local fault logs. This becomes essential for slashing, external audit, and incident response.

5. Tie DAG RPC outputs to **verifiable proofs** wherever possible so clients are not forced to trust a compromised RPC server blindly.

Deliverables and exit criteria: Archive and audit roles can replay the graph; pruning is safe and reproducible; DAG evidence is exportable; slashable behavior leaves durable proof.

Source note. Synergy_Network_Node_Technical_Specification.docx extracted lines 777-997 and 2916-2930; Synergy_Network_Security_Implementation_Specification.docx extracted lines 1540-1544.

Phase 7 - Make the Synergy Node Control Panel DAG-aware without making it dangerous

The Control Panel should become the official operational surface for DAG clusters, but it must do that in the same bounded, policy-first way already described in the node specification. No expert-mode free-for-all.

1. Create typed role overlays such as **validator-dag-beta**, **committee-dag-beta**, **archive-validator-dag**, and **audit-validator-dag**. DAG should be enabled through an environment-approved overlay, not through arbitrary manual key insertion.
2. Add DAG-specific health metrics and health states: frontier depth, highest certified round, missing-parent rate, availability-certificate latency, DAG fetch failures, ordering-cut latency, certificate fraud alerts, equivocation alerts, peer diversity score, and pruning lag.
3. Add guided workflows for: enabling DAG on a canary validator, rotating DAG-related parameters safely, exporting DAG evidence bundles, entering quarantine, resuming from quarantine, and rolling back to the last known-good non-authoritative mode.
4. Make Jarvis refuse unsafe combinations such as public metrics exposure, disabled provenance checks, plaintext key workflows, uncertified-ordering mode, or DAG enablement on an environment/profile that has not been approved.
5. Bind DAG operations to the existing **signed update**, **secret isolation**, and **least-privilege** posture. DAG should add observability and safety, not a new pile of privileged knobs.

Deliverables and exit criteria: Control Panel can configure and monitor DAG roles safely; DAG feature flags are environment-specific; operator workflows remain bounded; unsafe combinations are blocked by policy.

Source note. Synergy_Network_Node_Technical_Specification.docx extracted lines 2639-2759 and 2916-2930; Synergy_Wallet_Specification_v3.pdf extracted lines 153-158 and 173-173.

Phase 8 - Rollout through testnet-beta in controlled blast-radius stages

The rollout should be staged. A DAG feature that goes from whiteboard to full beta cluster deployment in one jump is asking for an avoidable incident.

1. Stage 1: **single-cluster lab run** with synthetic load, archive node, and audit node. Confirm certificate generation, missing-parent recovery, and deterministic ordering.
2. Stage 2: **two-cluster internal run** using bridge relays and checkpoint exchange. Confirm cross-cluster proofs and epoch boundary handling.
3. Stage 3: **adversarial simulation run** including equivocation, hidden subgraph attempts, data withholding, spam bursts, partition scenarios, bridge loss, and stale/replay injections.

4. Stage 4: **canary DAG enablement** on a small subset of beta validators while other clusters remain on the previous path. Confirm that quarantine and rollback procedures actually work.
5. Stage 5: **full internal beta DAG rollout** only after archive/audit replay matches live ordering and no critical or high security findings remain.
6. Stage 6: **public or wider operator exposure** only after the external review and attack-simulation program are completed and release provenance workflows are exercised end to end.

Deliverables and exit criteria: Each stage must have a clean rollback path, measured security gates, and evidence retention. The rule is simple: if safety evidence is incomplete, the rollout does not advance.

*Source note. Synergy_Network_Roadmap (1).docx extracted lines 137-177;
Synergy_Network_Security_Implementation_Specification.docx extracted lines 1810-1818.*

5. DAG-specific security enforcement plan

DAG networks are vulnerable in ways that traditional linear block pipelines are not. The right response is not to panic about DAGs in general; it is to refuse the insecure variants. Synergy should treat the DAG as a **certified cluster data plane** and systematically shut down the classic DAG attack classes at the object, policy, and operator layers.

Threat class	Primary controls	Enforcement posture
DAG equivocation (same validator emits conflicting vertices in the same round or sequence window)	Require one-author-one-vertex-per-round in beta, maintain per-author chains, sign every vertex, and let audit validators detect duplicates immediately.	Treat as equivocation . Start by mapping it to the existing validator equivocation severity. Immediate quarantine and evidence export.
Data withholding / ghost vertices / fake availability	Require full-batch fetch or durable local blob proof before issuing an availability receipt. Never let uncertified or unavailable vertices influence ordering.	Reject the vertex, count the event, and escalate to slashing if the pattern is deliberate or repeated.
Hidden sub-DAG / parasite-chain-style attack / private conflicting branch	Keep the DAG cluster-local, require availability certificates, make round/epoch expiry strict, and keep QC as finality so private branches never become authoritative merely by existing.	Discard hidden branches once stale, preserve evidence, and slash if they contain conflicting spend or certificate fraud.
Sybil and collusion inside clusters	Retain PoSy's dual quorum, Synergy Score weighting,	Use anti-cartel analytics and cluster-rotation policies to

	cooperation-count quorum, cartel analysis, and rotation. Add parent-diversity rules so clique subgraphs are easier to detect.	reduce future influence; slash when collusion evidence crosses policy thresholds.
Censorship or parent omission	Monitor missing-parent rate, proposer fairness, and repeated exclusion patterns. Audit validators should compute whether certified vertices are being systematically omitted from ordering cuts.	Persistent provable censorship becomes a slashable liveness sabotage event.
Spam and DAG bloat DoS	Per-validator byte budgets, max vertices per round, backpressure, certificate gating, bridge rate limits, and aggressive post-finality pruning.	Throttle first; penalize repeated abuse; quarantine if the node's behavior threatens cluster stability.
Replay and stale-vertex injection	Bind every envelope to cluster_id, epoch_id, round, sequence number, timestamp, and expiry. Reuse the existing ML-KEM session ordering and signature model.	Drop and log immediately. Repeated deliberate replay becomes a slashable protocol-deviation event.
Eclipse, bridge compromise, and network partition	Enforce peer diversity, minimum bridge redundancy, signed relay chains, and safety-over-liveness behavior. No cluster should finalize based on uncertain graph visibility.	Pause rather than finalize incorrectly. Quarantine bad bridges, rotate sessions, resume only after safe visibility is restored.
Ordering manipulation / MEV / frontrunning	Integrate the encrypted mempool into the DAG. Batches should remain encrypted until the protocol-defined ordering moment. Proposers must not get a pre-decryption advantage.	Any bypass of encrypted ordering is a release blocker, not a minor bug.
Operator and supply-chain	Keep signed updates,	Rollback, quarantine, signed

compromise	provenance checks, HSM or device-isolated key paths, least privilege, localhost-only admin surfaces, and bounded Control Panel workflows.	evidence retention, and incident narrative generation become mandatory.
------------	---	---

5.1 Security rules that should be treated as non-negotiable

- **No uncertified ordering.** A vertex without valid availability evidence does not exist for ordering purposes.
- **No new public ports by default.** DAG should ride on the existing authenticated inter-node surfaces wherever possible. Public services remain 443-only, metrics stay localhost-only, and KMS remains non-public.
- **No weakening of QC verification.** DAG optimization must never weaken the finality path.
- **No plaintext key UX.** DAG enablement must not create shortcuts around the Control Panel's secret-handling model.
- **No optimistic continuation after graph-integrity failure.** If the local node cannot establish safe graph visibility, it should degrade, quarantine, or stop authoring authoritative artifacts.

Source note. Master Network, Ports, and Nginx Configuration Manual.docx extracted lines 162-174 and 2731-2737; Synergy_Network_Node_Technical_Specification.docx extracted lines 2916-2930.

5.2 Security testing and assurance work that must accompany the rollout

1. **Property tests** for DAG invariants: acyclicity, uniqueness of author/round in beta mode, certificate validity, deterministic ordering, and epoch-boundary expiry.
2. **Adversarial simulation suites** for hidden subgraphs, equivocation, data withholding, replay, censorship, spam bursts, bridge loss, and partition behavior.
3. **Archive/audit replay tests** that reconstruct ordering cuts from stored DAG artifacts and prove they match live blocks.
4. **Formal reasoning or model checking** for the critical invariants: no uncertified vertex reaches ordering, no two conflicting ordering cuts can both finalize under stated assumptions, and epoch boundaries prevent stale graph influence.
5. **Third-party review expansion** so the security audit explicitly covers the DAG availability layer, ordering-cut verification, encrypted batch handling, and the interaction between DAG evidence and slashing.

Source note. Synergy_Network_Security_Implementation_Specification.docx extracted lines 1806-1818 already require adversarial testing, formal verification of protocol invariants, and third-party review before launch; DAG must be folded into that existing assurance program rather than treated as a side feature.

6. Inferred repository workstreams and the likely missing dependency

Because the two GitHub repositories were not retrievable in-session, the mapping below is necessarily inferred from the repository names and the product responsibilities described in the attached specifications. The key practical point is this: **these two repositories are unlikely to be sufficient on their own to implement validator-cluster DAG logic** unless one of them also contains the actual validator client or consensus engine. If the consensus runtime lives elsewhere, that third repo must be brought into scope immediately.

Workstream	What should live there
`testnet-beta`	Environment overlays; inventory and topology; feature flags; bridge assignments; canary rollout plans; DAG enablement/rollback scripts; metrics dashboards; attack-simulation harnesses; archive/audit node deployment profiles; updated documentation for topology, exposure, and release gates.
`synergy-node-control-panel`	Typed DAG role profiles; DAG health dashboards; quarantine and rollback flows; DAG evidence export; safe parameter validation; environment guardrails; signed-update gating; operator narratives and alerting.
Likely missing consensus/client repo	DagVertex/DagCertificate/OrderingCut objects; batch builder; availability receipts; parent-selection rules; proposer integration; validator verification; storage/pruning; slashing evidence generation; DAG RPC endpoints; archive/audit replay logic.

6.1 Concrete repository-level plan of attack

1. In **`testnet-beta`**, add environment-specific DAG overlays, canary topology definitions, checkpoint/bridge deployment rules, attack-test scenarios, and rollout/rollback playbooks. Also make sure no topology change casually exposes new public ports.
2. In **`synergy-node-control-panel`**, add DAG-aware role templates and DAG safety validations. Jarvis should be able to tell an operator exactly why a DAG profile is safe or unsafe in a given environment.
3. In the **actual consensus/client repo**, implement the protocol changes. That is where the real DAG work lives: data availability, certificates, deterministic cuts, verifier logic, storage, RPC, and slashable evidence.

4. Create a **cross-repo release checklist** so a DAG rollout cannot go live merely because deployment overlays are ready while verifier logic, archive retention, or operator rollback flows are still incomplete.

7. Release gates for enabling DAG in validator clusters

Gate	Must be true before rollout advances
Specification gate	ADR pack approved; cluster size and epoch duration canonicalized; DAG object schemas frozen.
Protocol safety gate	No uncertified vertex can enter ordering; QC verification path unchanged; conflicting finalized cuts impossible under tested assumptions.
Availability gate	Missing-parent fetch works; archive/audit nodes can reconstruct certified vertices; certificate fraud is detected.
Security gate	Adversarial simulations for equivocation, hidden subgraphs, replay, withholding, spam, bridge loss, and partition all pass agreed criteria.
Operational gate	Control Panel can enable, monitor, quarantine, rollback, and export evidence for DAG roles without unsafe escape hatches.
Exposure gate	No unauthorized public surfaces; metrics remain localhost-only; signed updates and provenance checks are enforced.
Audit gate	Third-party review completed with no unresolved critical or high findings in the DAG path.
Economic/accountability gate	DAG-specific infractions are slashing-addressable and evidence formats are stable.

8. Recommended 30/60/90-day execution sequence

Window	Primary outcomes
Days 0-30	Finish ADRs; resolve cluster-size and epoch-

	duration conflicts; identify consensus repo; define DAG objects, RPCs, and naming deltas; freeze invariant list; design archive/audit evidence model.
Days 31-60	Implement batch builder, vertex builder, availability receipts, certificates, DAG storage, basic metrics, and audit export; add DAG role overlays and safety checks in the Control Panel.
Days 61-90	Integrate proposer ordering cuts and validator verification; run adversarial simulations; light up canary clusters in testnet-beta; complete external review prep; lock rollout gates.

Appendix A - Proposed DAG RPC additions

- ``synergy_getClusterDagState(cluster_id)`` - return frontier summary, highest certified round, watermark, and health indicators.
- ``synergy_getDagVertex(vertex_id)`` - return the signed vertex envelope and proof references.
- ``synergy_getDagCertificate(vertex_id)`` - return availability-certificate details and signer bitmap/root.
- ``synergy_getDagFrontier(cluster_id)`` - return the currently orderable certified frontier.
- ``synergy_getOrderingCut(block_hash_or_qc_id)`` - return the cut commitment and included vertex set root.
- ``synergy_getDagEquivocations(epoch_id)`` - return detected same-author same-round conflicts and evidence handles.
- ``synergy_getDagAvailabilityLag(cluster_id)`` - return lag metrics for missing-parent and batch-fetch health.
- ``synergy_getValidatorDagActivity(address, epoch_id)`` - expose per-validator DAG participation and failure patterns.

Source note. These extend the current RPC surface, which presently exposes validator and general chain methods but not DAG observability. See `rpc-methods.md` extracted lines 167-274.

Appendix B - Proposed beta DAG configuration keys

- ``[dag] enabled = false``
- ``mode = "certified_mempool"``
- ``require_encrypted_batches = true``
- ``availability_threshold = "2f_plus_1"``

- `allow\_uncertified\_ordering = false`
- `max\_vertices\_per\_round\_per\_validator = 1`
- `min\_parent\_diversity = <committee-derived>`
- `vertex\_expiry\_rounds = <bounded>`
- `gc\_finality\_depth = <bounded>`
- `cross\_cluster\_mode = "checkpoints\_only"`
- `emit\_dag\_evidence = true`
- `quarantine\_on\_dag\_integrity\_failure = true`

Appendix C - DAG-specific slashing and accountability extensions

- `dag\_equivocation\_same\_round` - treat as validator equivocation severity; strong candidate to inherit the existing 15% slash posture used for equivocation in rewards/accountability documents.
- `dag\_certificate\_fraud` - forged or fraudulent availability certification; should be treated as severe slashable behavior.
- `dag\_data\_withholding` - repeated certified-but-unavailable batch behavior; escalating penalty.
- `dag\_censorship\_pattern` - provable systematic omission of eligible certified vertices from ordering cuts; escalating penalty.
- `dag\_replay\_or\_stale\_spam` - repeated stale/replay injection with malicious intent; lower-severity but escalating penalty.

Source note. The existing operator rewards and accountability material already includes strong penalties for equivocation and invalid state voting; DAG-specific categories should extend that framework rather than invent an unrelated one. See Synergy_Operator_Rewards_Standard_v1_0.docx extracted lines 329-341 and PoSy_updated.docx extracted lines 367-388.

Appendix D - Source basis used for this report

- PoSy_updated.docx - clustered architecture, bridge relays, dual quorum, QC finality, epoch rotation, PQ cryptography, and slashing.
- 03-proof-of-synergy-consensus.md - cooperative clustering rationale, deterministic finality, safety-over-liveness, and epoch-scoped accountability.
- Synergy_Network_Network_Configuration_Specification.docx - current beta/devnet parameters, ports, and validator topology.
- Master Network, Ports, and Nginx Configuration Manual.docx - exposure hard rules and firewall posture.
- Synergy_Network_Node_Technical_Specification.docx - node roles, Control Panel product model, trust boundaries, and hardening baseline.

- Synergy_Network_Security_Implementation_Specification.docx - encrypted mempool, state-proof verification, adversarial testing, formal verification, and audit requirements.
- Synergy_Protocol_Naming_Encoding_Standards.docx - identifier and algorithm conventions that need DAG extensions.
- rpc-methods.md - current RPC surface that will require DAG observability expansion.
- Synergy_Operator_Rewards_Standard_v1_0.docx - validator rewards and penalty posture that DAG accountability should extend.
- Synergy_Network_Roadmap (1).docx - explicit DAG validator cluster milestone and related security roadmap items.
- Synergy_Wallet_Specification_v3.pdf - authority separation and trust-boundary principles that should continue to shape Control Panel behavior.